Interpretable Joint-Time Error Localization for Rehabilitation Exercise Feedback: Midterm Report

Hyungeon, Uijin Si, Roshini, Joshi
Handong Global University
Pohang, Republic of Korea
SIT33007 AI Convergence and Applications

Abstract

Home-based rehabilitation exercises are usually performed without a therapist, yet most skeleton-based assessment models return only a correctness label or a single quality score, which does not tell the user what went wrong or when. This project targets a model that, as a skeleton sequence unfolds, predicts at every frame whether an error is occurring, which error type it is, and which body region is involved. Following the instructor's feedback on the proposal, the scope was changed from per-recording error classification to frame-level detection. This midterm report covers the work completed so far, which is concentrated on data preparation for the KERAAL low-back-pain rehabilitation dataset. We parsed the therapists' ANVIL annotations, verified that the skeleton streams and the annotation time axis share a 15 fps clock (the 10 fps value in the video metadata is unreliable), and built frame-level model-input tables for Kinect (841 recordings, 237,733 frames) and BlazePose (835 recordings, 224,912 frames) skeletons, with quality-control flags for annotation coverage and truncated skeleton files. Erroneous frames make up only 6.4% (Kinect) and 6.9% (BlazePose) of Group 1A/2A frames, and several error/body-part combinations have fewer than 150 frames. These findings motivate merging body parts into Spine and Arms, imbalance-aware losses, and strictly subject-disjoint evaluation. No model results are reported yet; the remaining plan covers kinematic features, frame-wise TCN/ST-GCN baselines and evaluation.

1. Introduction

Rehabilitation for low-back pain relies heavily on exercises that patients repeat at home. Without a therapist present, small deviations such as leaning the trunk during a rotation or lifting the arms asymmetrically can go unnoticed for weeks. Skeleton-based motion analysis is an attractive basis for automatic feedback: joint coordinates are compact, less sensitive to clothing and background than RGB video, and cheap enough to process on a CPU.

Most existing work on skeleton-based rehabilitation assessment, however, answers a coarse question: was this repetition correct, or what is its overall quality score [4, 5]? Recent work on KERAAL classifies the error type of a whole recording [3]. For a user trying to correct a movement, the more useful questions are *which body region* moved incorrectly and *at what moment*. Answering them requires labels that are dense in time, which few rehabilitation datasets provide.

Our proposal initially aimed at joint-time error localization with multi-task heads. During scoping it was narrowed to subject-independent error classification per repetition. The instructor's feedback was to return to a finer target: a model that, as the skeleton sequence evolves, reports the error type and body region at every second or frame. KERAAL [1, 2] makes this feasible because its therapist annotations (ANVIL files) contain start and end times for each error together with the affected body part.

Converting these annotations into reliable frame-level targets turned out to be the main technical work of the first half of the semester. The contributions of this midterm stage are: (i) an inventory of the KERAAL Group 1A/2A/3 file structure and annotation formats; (ii) verification of the common time base between annotations, skeleton streams and video; (iii) frame-level model-input tables for two skeleton sources (Kinect and BlazePose) with quality-control flags; and (iv) label statistics that directly shape the model and evaluation design. Section 2 reviews related work, Section 3 defines the task, Section 4 describes the data preparation and its findings, Section 5 presents the planned method, Section 6 lists open issues, and Section 7 gives the remaining schedule.

2. Related work

2.1. Rehabilitation movement datasets

UI-PRMD [5] provides Kinect and Vicon recordings of ten rehabilitation exercises by ten healthy participants, with correct and non-optimal executions. KIMORE [4] contains five exercises performed by healthy participants and people with motor dysfunction, together with clinician scores. Both support correctness or quality-score

prediction, but neither labels the body part and time span of an error. KERAAL [1] records three low-back-pain exercises (torso rotation, flank stretch and hiding face) performed by patients and healthy participants, captured with RGB video, Kinect skeletons and 2D skeletons from OpenPose and BlazePose, and annotated by therapists with error type, affected body part and error time span. We therefore use KERAAL as the primary dataset and keep UI-PRMD and KIMORE as optional auxiliary data.

2.2. Skeleton models and error feedback

Spatial-temporal graph convolutional networks (ST-GCN) [6] model the skeleton as a graph whose nodes are joints and apply convolutions over space and time; they are a standard lightweight backbone for skeleton action recognition. Marusic *et al.* [3] apply a skeleton-based transformer to KERAAL to classify error types and argue that such classifications can support feedback. Their output is still one label per recording; our project extends the target to per-frame error and body-region labels.

2.3. Temporal action segmentation

Predicting a label for every frame is closely related to temporal action segmentation. Temporal convolutional networks (TCNs) with dilated causal convolutions are competitive with recurrent networks on sequence modelling [8]. MS-TCN [7] stacks several dilated TCN stages and adds a smoothing loss that penalizes rapid changes between frames, reducing over-segmentation. Its frame-wise and segmental metrics (F1@{10, 25, 50}) are directly applicable to error localization, and we adopt them in Section 5.

3. Problem formulation

A recording is a skeleton sequence $X = (x_1, \ldots, x_T)$ with $x_t \in \mathbb{R}^{J \times C}$, where J is the number of joints (33 landmarks for BlazePose [9]) and C the number of per-joint channels (coordinates, plus derived kinematic features). At 15 frames per second, frame t corresponds to time t/15 s. For every frame the model predicts:

Error state $e_t \in \{0, 1, \ldots, K\}$, where 0 means no error and 1…K are therapist-defined error types. Because the meaning of *Error1/2/3* in KERAAL may differ between exercises, each class is defined as an (exercise, error type) pair.

Body region $r_t \in \{0, 1\}^R$, a multi-label vector over body regions. Given the label counts in Section 4.5, we start with R = 2 (Spine, Arms).

The professor's requirement that predictions follow the sequence as it evolves implies a streaming setting: the prediction for frame t should depend only on frames up to t (or a short look-ahead). We therefore treat causal models as the main configuration and offline (bidirectional) models as an upper reference.

4. Dataset and data preparation

4.1. KERAAL structure

KERAAL includes 12 participants with low-back pain and 9 healthy participants [1]. Recordings are organized in groups: Group 1A (249 recordings) and Group 2A (51 recordings) carry body-part and time-span annotations, Group 3 (540 recordings) carries error labels without time spans, and Groups 1B/2B contain recordings without localization labels. In the downloaded data, each of the Group 1A/2A folders contains the sub-folders *annotatorA*, *annotatorB*, *blazepose*, *kinect*, *openpose* and *videos*. The annotations are ANVIL XML files, stored partly as hidden *.anvil* files; annotatorA files are encoded in UTF-16 and annotatorB files in UTF-8, so the parser detects the encoding per file. Participants are coded R1–R6 in Group 1A, S1–S6 in Group 2A and P1–P3 in Group 3. Figure 1 summarizes the preparation pipeline.

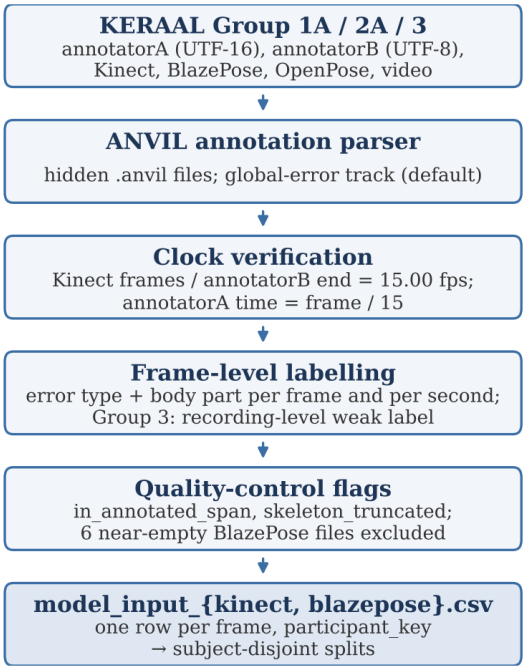


Figure 1. Data preparation pipeline from raw KERAAL files to frame-level model-input tables.

4.2. Establishing a common time base

Frame-level labels are only meaningful if annotation times map exactly onto skeleton frames, so we verified the clock before building any labels.

Frame rate. For every recording, dividing the number of Kinect frames by the end time of the annotatorB file gives exactly 15.00. The video containers report 10 fps, but their frame counts equal the Kinect frame counts, so we treat the container metadata as wrong and use 15 fps

throughout. In two Group 2A recordings, the video has fewer frames than the Kinect stream.

AnnotatorA time axis. In error-free recordings, the end time of annotatorA equals that of annotatorB to two decimal places, confirming that annotatorA times follow frame/15.

AnnotatorB. AnnotatorB labels the entire recording (from 0 to the end) as a single segment. It is therefore useful for the clock check above but unsuitable as a source of frame-level labels; all temporal labels come from annotatorA.

4.3. Frame-level label construction

Each annotatorA interval of the *global error* track is converted into frame indices using the 15 fps clock, and every frame inside an interval receives the corresponding error type and body part. Labels are stored both per frame and per second. The parser can switch to the *temporal error* track (option --interval-track temporal), because it is not yet confirmed which of the two tracks is closer to the actual error timing (Section 6).

AnnotatorA often stops annotating at the end of the last error rather than at the end of the recording. Frames after that point are not known to be correct, so each frame carries the flag **in_annotated_span** (yes/no), and training uses only frames with *yes*. Group 3 recordings receive a recording-level weak label (the same label on every frame) because they have no time spans.

The outputs are two tables, *model_input_kinect.csv* and *model_input_blazepose.csv*, with one row per frame and a *participant_key* column for subject-level splitting, plus a text QC report for each.

4.4. Quality control

Of the BlazePose JSON files, 115 recordings are truncated partway through (mostly Group 1A participants R4 and R6), and 6 are nearly empty and were excluded. Truncated recordings are kept but flagged with **skeleton_truncated**, so that they can be filtered or labelled only up to the truncation point. This explains the difference in recording counts between the two skeleton sources (Table 1).

TABLE 1. SIZE OF THE FRAME-LEVEL MODEL-INPUT TABLES. ERROR-FRAME RATIO IS COMPUTED OVER GROUP 1A/2A FRAMES.

Skeleton source	Recordings	Frames (rows)	Error frames
Kinect	841	237,733	6.4%
BlazePose	835	224,912	6.9%

4.5. Label statistics

Table 2 and Figure 2 give the error-frame counts per error type and body part inside the annotated span of Groups 1A/2A (Kinect). Three observations follow.

TABLE 2. ERROR FRAMES BY ERROR TYPE AND BODY PART (KINECT, GROUPS 1A/2A, IN_ANNOTATED_SPAN = YES).

Error type	Body part	Frames	Share of type
Error1	Both arms	1,877	87.3%
	Left arm	140	6.5%
	Right arm	132	6.1%
Error2	Spine	3,476	97.7%
	Right arm	83	2.3%
Error3	Both arms	1,148	100%
Total		**6,856**	

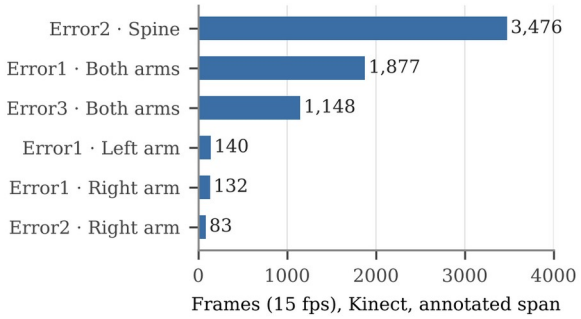


Figure 2. Error-frame counts per (error type, body part) combination. Single-arm combinations have fewer than 150 frames each.

Severe class imbalance. Only about 6–7% of frames contain an error. A model that always predicts “no error” would reach over 93% frame accuracy, so accuracy is not an informative metric, and the loss must be imbalance-aware.

Body part is largely determined by error type. Error2 is almost always Spine (97.7%), and Error1 and Error3 are almost always Both arms. The single-arm classes have only 83–140 frames. We therefore merge body parts into Spine and Arms for the main experiments. This also means that a separate body-region head may add little beyond the error-type head on KERAAL, which must be taken into account when testing the multi-task hypothesis from the proposal.

Uneven coverage across participants. Participants S4 and S5 (Group 2A) have no error frames at all, and Error3 appears only for participants R2, R3 and R6. Under leave-one-participant-out evaluation, Error3 recall can therefore be measured on only three folds, and per-class results must be reported with the number of participants that contribute to them.

4.6. Evaluation split

All primary results will use subject-disjoint splits: leave-one-participant-out or GroupKFold over *participant_key* on the 12 Group 1A/2A participants.

Normalization statistics, class weights and decision thresholds are computed from training participants only. Group 3 may be used for weak-label pre-training, but never with participants that also appear in a test fold.

5. Planned method

5.1. Preprocessing and kinematic features

Skeletons are root-centred at the pelvis or spine joint, scaled by shoulder or hip width, and rotated to a torso-aligned frame. Missing or low-confidence joints are masked. On top of coordinates we compute velocities and accelerations, joint angles relevant to the three exercises (trunk flexion, lateral bending and axial rotation; shoulder elevation; elbow flexion) and left–right symmetry features. Comparing coordinates alone with coordinates plus kinematic features is one of the planned ablations.

5.2. Models

We will compare, under identical splits and for both skeleton sources: (i) a causal LSTM; (ii) a causal dilated TCN [8], and a two-stage MS-TCN-style refinement [7]; and (iii) a lightweight ST-GCN [6] encoder followed by a temporal head. Each model has an error-state head (softmax over K + 1 classes) and a body-region head (sigmoid over R regions). Training minimizes

$$L = L_{err} + \lambda_r L_{reg} + \lambda_s L_{smooth} \quad (1)$$

where L_err is a class-weighted cross-entropy or focal loss [10] over frames, L_reg is a binary cross-entropy over regions computed on error frames, and L_smooth is the truncated mean-squared smoothing loss of MS-TCN [7], which discourages fragmented predictions. The weights λ are tuned on validation participants only.

5.3. Metrics

We report frame-wise macro-F1 over error classes (excluding the dominant no-error class from the average as well as reporting it separately), segmental F1@{10, 25, 50} [7], onset and offset error in seconds, region macro-F1 on error frames, and recording-level error detection. Results are reported per participant with mean and spread over at least three random seeds. CPU latency per frame will be measured for the best causal model.

6. Open issues

Group 3 body-part codes. The mapping of *BodypartIndex1–3* in Group 3 to actual body parts has not been confirmed.

Global vs. temporal error track. It is not yet confirmed which annotatorA track better matches the true error timing. Global is used by default; we will compare both on a sample of recordings against the videos.

Error semantics per exercise. Error1/2/3 may denote different movements in different exercises, so we will treat (exercise, error type) pairs as classes until the taxonomy is checked against the dataset documentation.

Video–skeleton mismatch. Two Group 2A recordings have fewer video frames than Kinect frames, which affects BlazePose rows derived from the video. These recordings will be inspected individually.

Analysis plan. The team still has to confirm that using Group 3 is consistent with the analysis plan that was fixed before looking at Group 3 results.

7. Remaining schedule and roles

Table 3 lists the plan for the second half of the semester. Roles follow the proposal: Hyungeon leads models, splits and statistics; Uijin leads preprocessing, kinematic features, baselines and CPU benchmarking; Roshini reviews the error taxonomy, body-region mapping and feedback wording; Joshi maintains the data inventory, references, experiment logs and annotation QA.

TABLE 3. PLAN FOR THE REMAINING WEEKS.

Week	Goal	Deliverable
8	Midterm; freeze preprocessing and features	This report, feature module
9	Causal LSTM / TCN on Kinect and BlazePose	Baseline results (LOSO)
10	MS-TCN-style refinement, ST-GCN, region head	Multi-task results
11	Ablations, error analysis, CPU latency	Final tables and figures
12	Final report and presentation	Code, report, demo

8. Conclusion

In the first half of the project we redefined the task as frame-level error and body-region detection and built the data foundation it requires. The main finding is that KERAAL's annotations can be aligned to skeleton frames on a 15 fps clock, but that the resulting labels are sparse (6–7% error frames), dominated by a few error/body-part combinations, and unevenly spread across participants. These properties set the next steps: merged body regions, imbalance-aware and smoothed training objectives, causal temporal models, and subject-disjoint evaluation that reports per-participant results.

References

[1] S. M. Nguyen, M. Devanne, O. Remy-Neris, M. Lempereur, and A. Thepaut. A medical low-back pain physical rehabilitation dataset for human body movement analysis. In Proc. IJCNN, 2024. arXiv:2407.00521. https://arxiv.org/abs/2407.00521

[2] S. M. Nguyen et al. KERAAL dataset repository. https://github.com/nguyensmai/KeraalDataset

[3] A. Marusic, S. M. Nguyen, and A. Tapus. Skeleton-based transformer for classification of errors and better feedback

in low back pain physical rehabilitation exercises. arXiv:2504.13866, 2025. https://arxiv.org/abs/2504.13866

[4] M. Capecci, M. G. Ceravolo, F. Ferracuti, et al. The KIMORE dataset: KInematic assessment of MOvement and clinical scores for remote monitoring of physical REhabilitation. IEEE Trans. Neural Syst. Rehabil. Eng., 27(7):1436–1448, 2019. https://vrai.dii.univpm.it/node/204704

[5] A. Vakanski, H. Jun, D. Paul, and R. Baker. A data set of human body movements for physical rehabilitation exercises. Data, 3(1):2, 2018. https://www.mdpi.com/2306-5729/3/1/2

[6] S. Yan, Y. Xiong, and D. Lin. Spatial temporal graph convolutional networks for skeleton-based action recognition. In Proc. AAAI, 2018. https://arxiv.org/abs/1801.07455

[7] Y. Abu Farha and J. Gall. MS-TCN: Multi-stage temporal convolutional network for action segmentation. In Proc. CVPR, 2019. https://arxiv.org/abs/1903.01945

[8] S. Bai, J. Z. Kolter, and V. Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv:1803.01271, 2018. https://arxiv.org/abs/1803.01271

[9] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann. BlazePose: On-device real-time body pose tracking. arXiv:2006.10204, 2020. https://arxiv.org/abs/2006.10204

[10] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. In Proc. ICCV, 2017. https://arxiv.org/abs/1708.02002